

Unit

3

Lesson

2



Beyond Imagination.

Break Language Barriers.

Time Frame: [80-90 min].

Ready, Set, Learn!

By the end of this lesson students will be able to:

- Connect a smart glove to a mobile device and display or speak translated messages.
- Code a translation app using MIT App Inventor with Text-to-Speech and Bluetooth features.
- Explain how technology can support people who speak different languages or have different communication needs.

Challenge questions of the lesson.

- How can a smart glove be connected to a mobile device to display or speak translated messages?
- What coding blocks and components in MIT App Inventor are essential for building a translation app with Text-to-Speech and Bluetooth?
- Why is it important to include both visual and audio outputs in an app designed for users with different communication needs?
- How can technology like a smart glove and translation app help bridge language barriers and support people who speak different languages?
- What challenges might arise when coding Bluetooth communication, and how can they be addressed?

SEL Relation

- **Self-Awareness:** Students reflect on how their design choices can impact communication for diverse users, fostering empathy and understanding of others' needs.
- **Self-Management:** The lesson develops students' ability to focus on complex coding tasks, manage challenges, and persist through debugging and problem-solving.
- **Social Awareness:** By exploring language barriers and assistive technology, students cultivate empathy and respect for individuals with different communication needs and cultural backgrounds.
- **Relationship Skills:** Collaborative app design and peer feedback activities promote teamwork, effective communication, and constructive criticism.
- **Responsible Decision-Making:** Students make thoughtful design and coding decisions that prioritize accessibility and inclusion, understanding the broader social impact of technology.

Engage



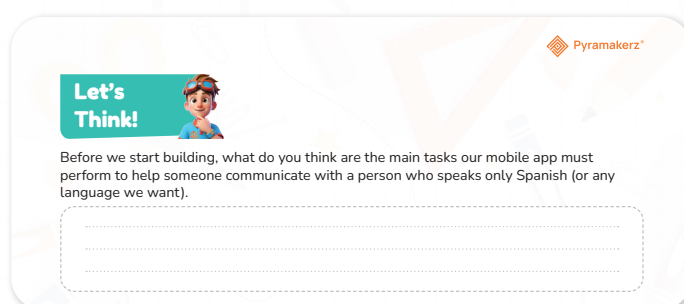
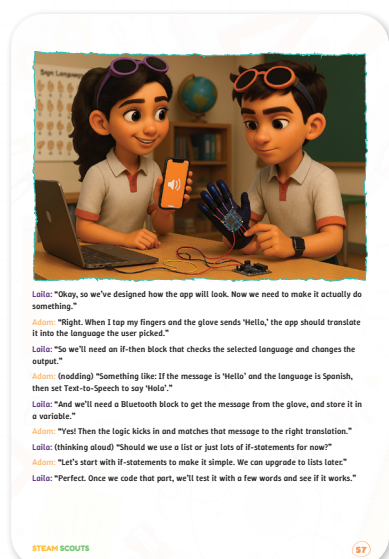
Teacher's Role:

- Gather students and create a welcoming environment to spark curiosity.
- Read the story aloud with expression, emphasizing problem-solving and teamwork.
- Use open-ended questions to connect students' prior knowledge about language and communication.
- Facilitate discussion around the challenges of communication across languages.
- Prompt students to think critically about the app's required functions before building it.
- Encourage students to share ideas and listen actively to peers' thoughts.

Expected from Students:

- Listen attentively to the story, noting key ideas about app design and language translation.
- Reflect on challenges faced by people who speak different languages.
- Participate actively in the think activity by writing or sharing their ideas.
- Collaborate respectfully during group discussions.
- Begin to form ideas about how technology can help break language barriers.

Parts of the book included.



Story



Teacher's Role:

- **Set the stage:** Gather students and create an inviting, focused atmosphere.
- **Read aloud:** Use a clear, engaging voice with expression to highlight the teamwork between Adam and Laila as they design the app.
- **Emphasize key ideas:** Pause slightly to stress important terms like “if-then block,” “Bluetooth,” and “logic.”
- **Prompt curiosity:** Ask students to think about how apps can translate languages and why conditional logic might be helpful.
- **Connect to prior knowledge:** Briefly recall earlier lessons on coding logic and Bluetooth communication.
- **Encourage predictions:** Invite students to imagine how the app might work once coded.
- **Check comprehension:** Ask guiding questions to confirm understanding and prepare for the next activity.

Possible Strategies for Implementation:

- **Story Visualization:** Encourage students to imagine the scene vividly, picturing Adam and Laila working on the app.
- **Think-Pair-Share:** After reading, have students discuss with a partner how conditional logic helps apps work.
- **Guided Questions:** Ask questions such as:
 - “What does an ‘if-then’ statement do in a program?”
 - “Why might Bluetooth be important for this glove app?”
 - “How can programming help break language barriers?”
- **Relate to Real Life:** Share examples of apps that translate languages to make the concept relatable.
- **Use Vocabulary Anchors:** Write key terms on the board and refer back to them during the story.
- **Check for Engagement:** Monitor students’ body language and responses to maintain active participation.

Let's Think!



Objective:

Students will identify and articulate the main functions a mobile app must perform to enable communication between the smart glove and a user who speaks a different

language. They will engage in critical thinking about app design requirements and prepare to translate these ideas into coding logic.

Answers:

1. The app needs to receive messages sent from the glove.
2. It should detect which language the user has selected.
3. The app should translate the message into the selected language accurately.
4. It needs to display or speak the translated message clearly.
5. The app might include buttons or icons to choose languages.
6. It should be easy to use for people with communication challenges.

Possible Strategies for Implementation:

- **Activate Prior Knowledge:** Begin by asking students to recall earlier lessons about the glove's messages and how the app will receive them.
- **Think-Pair-Share:** Allow students to first think individually, then discuss with a partner, and finally share ideas with the whole class.
- **Prompt with Guiding Questions:**
 - "What tasks should the app do when it receives a message from the glove?"
 - "How will the app know which language to translate into?"
 - "What features might help users understand the message clearly?"
- **Visual Brainstorming:** Use a whiteboard or shared document to list students' ideas, grouping similar tasks or features.
- **Encourage Specificity:** Ask students to provide concrete examples, such as translating words or playing audio.
- **Connect to Real-World Apps:** Reference translation apps students may have used to ground their thinking.
- **Clarify Vocabulary:** Explain or review terms like 'main tasks,' 'communication,' and 'language translation' as needed.

Explorer's Toolkit



Teacher's Role:

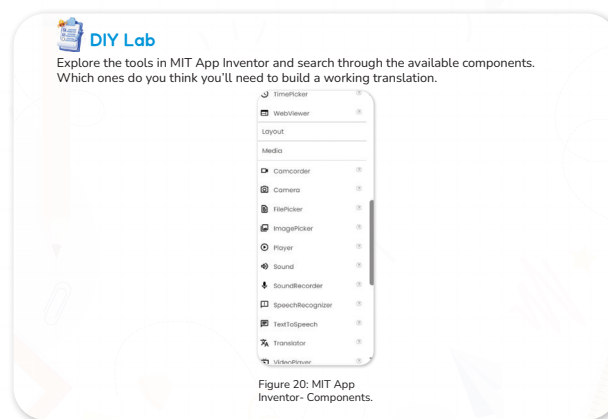
- Guide students in navigating MIT App Inventor's interface and components panel.
- Encourage curiosity and exploration, prompting students to think about each component's purpose.
- Support students by answering questions or demonstrating how to find and examine components.
- Facilitate small group or pair discussions on which components might be useful for the translation app.

- Help students make connections between the app's functional requirements (from the Think activity) and the available components.
- Monitor student engagement and provide feedback to deepen understanding.

Expected from students:

- Actively explore the MIT App Inventor interface and components list.
- Identify and note components that could help build a translation app, such as text input, buttons, language selectors, speech recognizers, and text-to-speech blocks.
- Participate in discussions or share insights about component functions and possible uses.
- Ask questions and seek clarification when needed.
- Reflect on how each component might contribute to the app's goals.

Parts of the book included.



DIY Lab

Objective:

Students will explore the components and tools available in MIT App Inventor to identify those necessary for building a translation app. This exploration will help them understand the app-building environment and the features they can use to create an effective communication tool.

Answer: Translator

Possible Strategies for Implementation:

- **Demonstration:** Begin with a short demo of the MIT App Inventor workspace, showing where to find components.
- **Guided Exploration:** Provide students with a checklist or worksheet prompting them to look for certain types of components (e.g., media, sensors, user interface).
- **Think-Pair-Share:** Have students first explore individually, then discuss their findings with a partner or group.
- **Component Matching:** Ask students to match app needs identified in the Think activity with possible components (e.g., text-to-speech for output, buttons for language selection).

- **Reflection Prompt:** End with a group reflection on which components are most essential and why.
- **Use of Visual Aids:** Provide screenshots or videos to reinforce component functions.
- **Scaffolded Support:** Provide hints or examples for less familiar components like Bluetooth or SpeechRecognizer.

Explain



Teacher's Role:

- Prepare students by setting clear expectations before watching the video.
- Encourage active viewing: prompt students to pay close attention to key parts such as component setup, programming logic, and data flow.
- Pause the video at critical moments to check for understanding and ask guiding questions (e.g., “What is happening here?” “How does the app get the translated text?”).
- Support students in connecting the video content to their own project and design ideas.
- Clarify any technical terms or processes that may be new to students.
- Facilitate a brief discussion or Q&A after the video to reinforce learning and address confusion.

Expected from students:

- Watch the video attentively, focusing on how translation is implemented in MIT App Inventor.
- Take notes or mark points of interest related to components, programming blocks, or translation flow.
- Ask questions when parts of the video are unclear.
- Participate in discussions or reflections prompted by the teacher post-video.
- Begin thinking about how to apply the concepts to their own app design.

Parts of the book included.



Screen Lessons

Tap the link to explore how translation works in MIT App Inventor.

<https://www.youtube.com/watch?v=i6RclBbitBY>



58

Screen Lessons

Objective:

Students will understand how translation works in MIT App Inventor by watching a tutorial video. They will learn how to design and program components that enable language translation within an app, including how data flows between user inputs and translation outputs.

Video Link: <https://www.youtube.com/watch?v=i6RclBbitBY>



Possible Strategies for Implementation:

- **Pre-Viewing Discussion:** Ask students what they expect to learn from the video about translation apps.
- **Guided Viewing:** Play the video in segments, pausing after important sections to review or explain.
- **Question Prompts:** Prepare guiding questions related to the video to prompt active thinking and engagement.
- **Note-Taking:** Encourage students to jot down key points, especially about how user inputs are processed and translated.
- **Group Reflection:** After viewing, have students share their understanding in pairs or small groups, comparing notes and insights.
- **Link to Design:** Relate the video content back to their own app-building plans and interface designs.
- **Technical Clarification:** Provide extra explanations or demos on specific blocks or translation logic if needed.
- **Follow-up Activity:** Assign a short quiz or discussion to check comprehension of video content.

Elaborate



Teacher's Role:

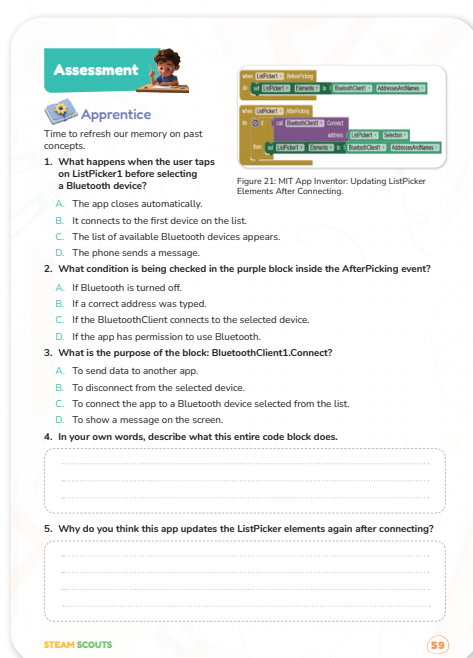
- Facilitate student collaboration during the Builder activity to ensure understanding of Bluetooth connection and app functionality.
- Monitor student progress on creating the Bluetooth Error Handler and Timer Block in MIT App Inventor.
- Prompt students to troubleshoot issues by asking guiding questions about their code logic and app responses.
- Encourage students to test their translation app thoroughly during the Showcase phase.
- Lead a reflection discussion on the importance of real-time connection feedback and its impact on user experience.

- Provide timely, constructive feedback on both the technical and design aspects of student projects.

Expected from students:

- Actively participate in building the Bluetooth status checker and timer blocks in MIT App Inventor.
- Collaborate with peers to solve programming challenges and debug errors.
- Test the app's connection status feedback by simulating connection and disconnection.
- Demonstrate understanding by explaining the purpose of each block used in their code.
- Reflect on the functionality and usability of the translation app in the Showcase.
- Provide feedback on peers' apps and receive constructive criticism to improve their own work.

Parts of the book included.



Apprentice

Objective:

Students will demonstrate understanding of Bluetooth connection management in MIT App Inventor by identifying the functions of key blocks and explaining how the app handles device selection, connection status, and permission checks. They will apply this knowledge to interpret code behavior and describe how the app updates its interface based on Bluetooth connectivity.

Answers:

1. What happens when the user taps on ListPicker1 before selecting a Bluetooth device?

- Answer:
D. The list of available Bluetooth devices appears.
2. What condition is being checked in the purple block inside the AfterPicking event?
- Answer:
D. If the app has permission to use Bluetooth.
3. What is the purpose of the block: BluetoothClient1.Connect?
- Answer:
C. To connect the app to a Bluetooth device selected from the list.
4. In your own words, describe what this entire code block does.
- Answer:
• This code connects the app to a Bluetooth device chosen by the user, checks for permissions, and updates the app based on the connection status.
5. Why do you think this app updates the ListPicker elements again after connecting?
- Answer:
• To refresh the list and show the current connection status or available devices accurately.

Possible Strategies for Implementation:

1. Pre-Assessment Review

- Quickly review key Bluetooth blocks (e.g., BluetoothClient1.Connect, AfterPicking, BluetoothClient1.Connect) before starting the quiz to refresh student memory.

2. Guided Questioning

- Ask students to explain the purpose of the Bluetooth connection components in their own words before attempting the quiz.

3. Quiz Engagement

- Use the multiple-choice questions as a formative quiz either individually or in pairs to promote discussion.

4. Feedback and Clarification

- Review quiz answers as a group, clarifying any misunderstandings about Bluetooth connection handling and event-driven programming.

5. Reflection

- Have students summarize what happens when the app connects/disconnects and how the UI responds to these events.



Builder

Objective:

Students will learn to program a visual Bluetooth connection checker in MIT App Inventor.

They will apply conditional logic blocks to handle different device connection statuses and display real-time updates and messages on the app screen.

Activity instructions:

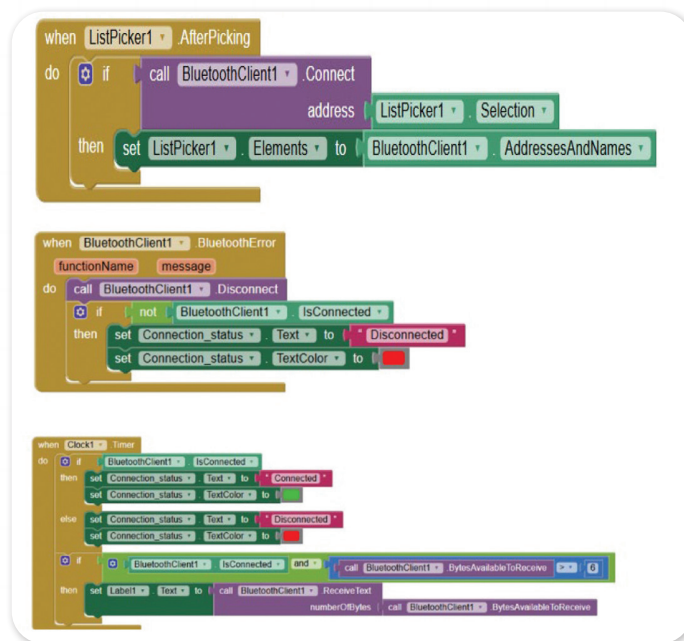
1. Open MIT App Inventor
 - Guide students to log in to MIT App Inventor and open their existing glove app project or start a new one.
2. Add UI Elements
 - Add a Label and name it Connection_status.
 - Set default text to “Checking...”
 - Increase font size and make the text bold.
 - Add a second Label named Label1 for displaying received messages.
3. Add Bluetooth Client & Clock.
 - From the “Connectivity” drawer, drag BluetoothClient to the workspace.
 - From the “Sensors” drawer, add a Clock component. Set TimerInterval to 1000 (1 second).
4. Create Bluetooth Error Handler.
 - Drag a when BluetoothClient.BluetoothError block.
 - Inside it:
 - Add call BluetoothClient.Disconnect.
 - Add if not BluetoothClient.IsConnected → set Connection_status.Text to “Disconnected” and color to red.
5. Build the Timer Block
 - Drag a when Clock.Timer block.
 - Inside it:
 - If BluetoothClient.IsConnected, set Connection_status.Text to “Connected” and color to green.
 - Else, set it to “Disconnected” and color to red.
 - Then, if connected and BytesAvailableToReceive > 6:
 - Set Label1.Text to BluetoothClient.ReceiveText.
6. Test the App
 - Ask students to connect their phones via Bluetooth to the glove circuit.
 - Observe how the connection status updates live, and if data is received, how it appears.

Possible Strategies for Implementation:

- **Model Before Release:** Show students a working version of the status checker. Walk through the logic in the blocks before letting them build.

- **Color Code Logic:** Emphasize why green is used for “Connected” and red for “Disconnected”—relating to UI clarity and accessibility.
- **Think-Aloud Debugging:** If something doesn’t work, model debugging strategies (e.g., “Hmm, let’s check if the clock is enabled.”).
- **Pair Programming:** Have students work in pairs. One builds the UI while the other builds the logic blocks, then switch roles.
- **Anchor to Real Use:** Ask, “Why is this important for someone using the glove?” Encourage students to imagine how confusing it would be to not know if the device is working or connected.
- **Use Visual Prompts:** Display the MIT App Inventor blocks on a projector or screen so students can refer while building.

Code Blocks:



Evaluate



Teacher's Role:

- **Technology Facilitator**
 - Guide students in navigating MIT App Inventor, specifically focusing on how to use the Translator component.
 - Ensure students understand how to connect blocks correctly and input parameters like language codes and labels.
- **Encourager of Problem-Solving**
 - Support students as they test their translation feature—ask guiding questions like “What happens when you change the input?” or “What should happen if the button is clicked?”

- **Promoter of Iteration**
 - Encourage students to test and revise their code if the app doesn't work as expected.
 - Help them troubleshoot by isolating errors in logic or block connections.
- **Connector to Real-World Relevance**
 - Emphasize the importance of this feature in supporting multilingual communication and accessibility.

Expected from Students:

- **Build the Translation Function**
 - Accurately connect the required blocks in MIT App Inventor to enable the translation feature.
 - Use appropriate labels, inputs, and outputs for a working translation interaction.
- **Test Their Code**
 - Run the app to ensure the translation feature works from text input to output.
 - Debug any issues and revise their code accordingly.
- **Understand the Code Logic**
 - Explain the purpose of each block (e.g., what the "Translator1.RequestTranslation" block does).
 - Demonstrate that they understand how user input triggers the translation and how results are displayed.
- **Reflect on Purpose**
 - Be ready to explain how this feature improves accessibility for people who speak different languages or cannot communicate easily.



Showcase

Objective:

Students will implement a translation feature in MIT App Inventor by using the Translator component. They will connect input from the glove or label text to enable automatic translation and test their app to ensure messages are correctly converted and displayed.

Activity Instructions:

1. Prepare the UI
 - Ensure students have a screen with:
 - A Label1: to display the text to be translated.
 - A Translated_text label: to show the translated output.
 - A Button1: labeled "Translate".
2. Add Translator Component
 - From the "Connectivity" section, drag and drop the Translator component into the

Designer (non-visible).

3. Set Button Click Logic

- In the Blocks tab:
 - Drag when Button1.Click.
 - Inside it, call Translator.RequestTranslation.
 - Set languageToTranslateTo as a language code (e.g., "it" for Italian).
 - Set textToTranslate as Label1.Text.

4. Get Translated Text

- Use the block when Translator.GotTranslation.
 - Inside it, set Translated_text.Text to get translation.

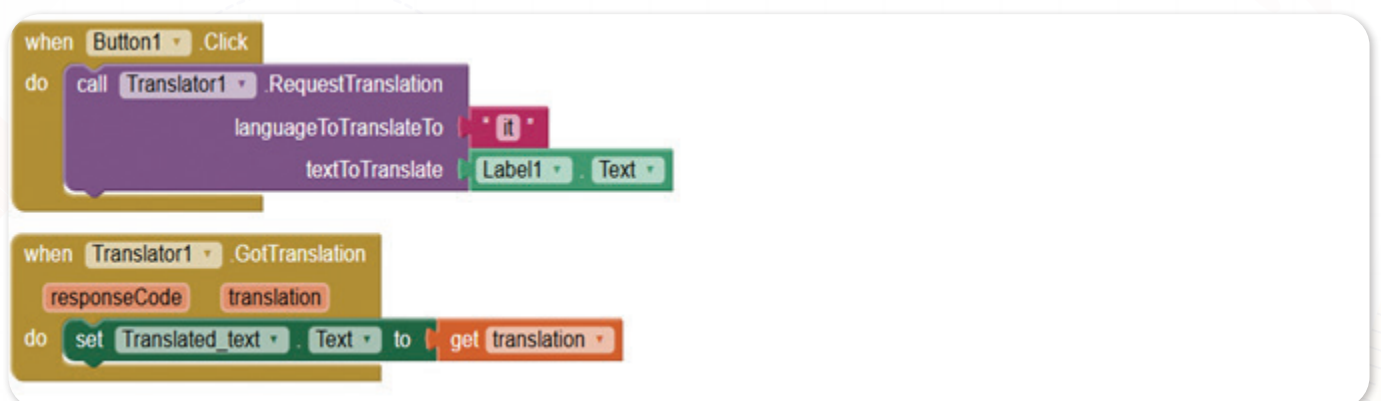
5. Test Your App

- Ask students to simulate a message appearing in Label1.
- Click “Translate” and observe the translated result.
- Try with multiple messages and language codes for deeper understanding.

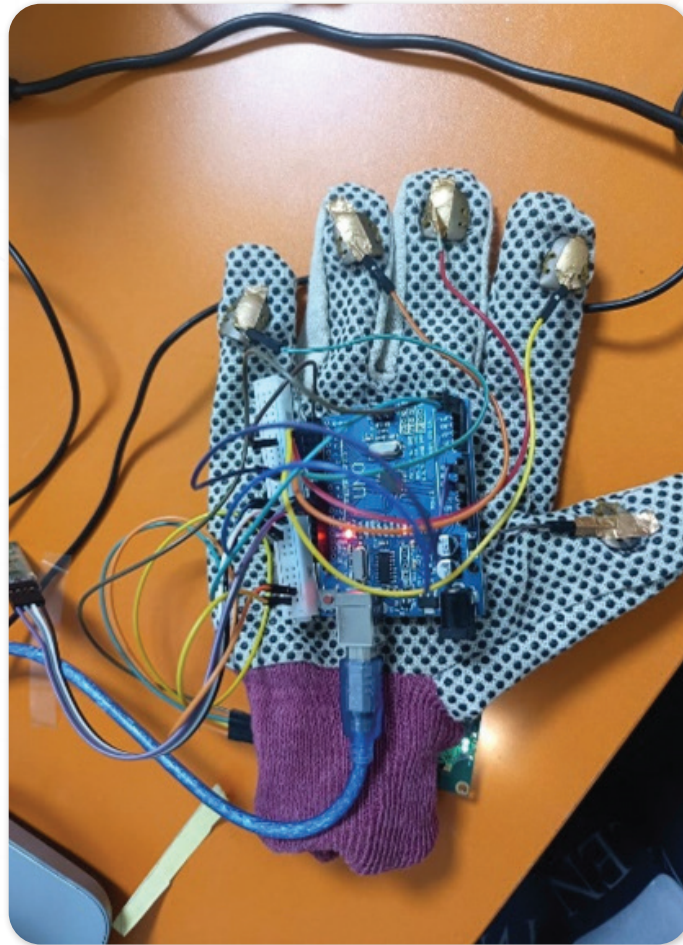
Possible Strategies for Implementation:

- **Language Code Exploration:** Provide a small chart or let students research language codes (like “es” for Spanish, “fr” for French).
- **Think-Pair-Share:** Have students predict what each block does, build in pairs, then share what worked and what didn’t.
- **Debugging Prompt:** If translation doesn’t work, ask guiding questions like:
 - “Did you set a language code?”
 - “Is your label text blank?”
 - “Did you connect the translation result to the right label?”
- **Real-Life Scenarios:** Ask students, “Where would this feature help someone?” (E.g., travel, schools, hospitals).
- **Self-check Prompt:** Encourage students to ask themselves, “Can my app show a translated word after a button click?” to reflect on function success.

Code Blocks:



Final Prototype:



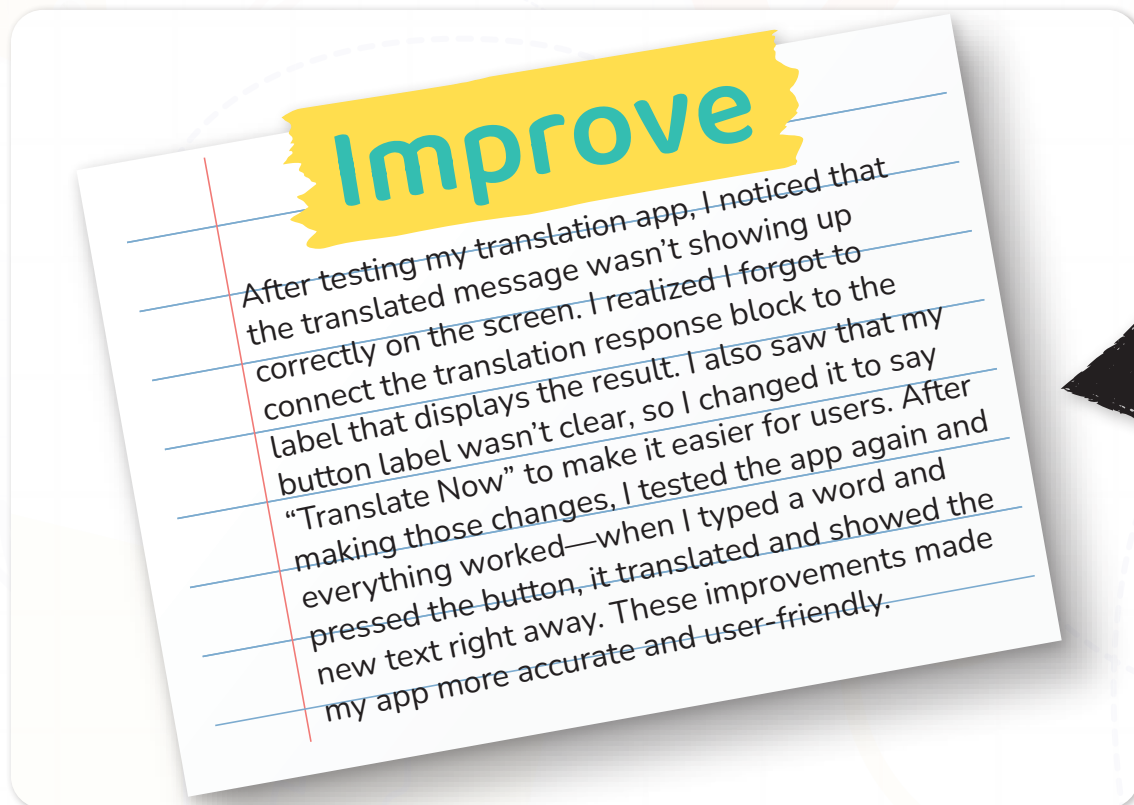
Final Prototype video: https://drive.google.com/drive/folders/179Z2EupAzCpWZ-_5ypjTlfmrAutAlAH5?usp=sharing



Rubric – Translation Feature in MIT App Inventor

Criteria	Excellent (4 Point)	Good (3 Points)	Needs Improvement (2 Points)	Incomplete (1 Point)
App Functionality	App translates text accurately and consistently with no errors.	App translates with minor errors or delays.	App works inconsistently or only partially translates.	App doesn't translate or crashes.
Block Logic Setup	All code blocks are correctly placed and fully functional.	Most blocks are placed correctly with one small error.	Multiple errors in block logic affect functionality.	Blocks are incomplete or missing.
User Interface Design	Layout is clean, organized, and user-friendly (clear buttons and labels).	Interface is mostly organized and usable.	Interface is confusing or cluttered.	Interface is missing key elements.
Testing & Debugging	Student thoroughly tested and fixed all errors independently.	Student tested and fixed most errors.	Student attempted testing but missed major bugs.	No testing or debugging done.
Creativity & Effort	Student added thoughtful touches (e.g., icon, colors, extra features).	Student followed instructions and completed required parts.	Student rushed through or copied others.	Little effort shown or incomplete.
Presentation & Explanation	Student clearly explained how the app works and what the blocks do.	Student gave a basic explanation of app and blocks.	Student gave a partial or unclear explanation.	Student didn't present or explain.

Don't forget to go to the improve part and add it to the improve part.



Final Scene (Story).

- **Set a Reflective and Celebratory Tone.**
 - The teacher gathers the class and reads the final story scene aloud with pride and excitement. After reading, the teacher pauses and says:
 - "You just created something incredible—an app and glove that let people understand each other without saying a single word. You didn't just code or build—you solved a real problem. That's the heart of global communication and empathy in action."

Possible Strategies for Implementation:

1. Celebration Circle – “My Voice, My Way”

- Gather students in a circle. Ask reflection questions like:
 - “What message did your device help share without words?”
 - “How would your solution help someone who can't speak or hear?”
 - “What did you learn about how tech can connect people?”

2. Communication Hero Certificates

- Give each student a certificate (e.g., “Global Communicator – For Making the World More Inclusive Through Technology”).

3. Hands-On Demo Day

- Let students present their final prototype (glove + app). Invite another class, parents, or even students who speak different languages to try them out.

4. "A Message to the World" Writing Activity.

- Ask students to write a short message they would send to someone in another country or someone with a hearing/speaking challenge—using their app or glove.
- Let me know if you'd like a printable certificate design or a slide template for the closing activity!



Now I Can

Objective:

Students will reflect on their learning and identify how they've grown in understanding communication, empathy, and the role of assistive tools in helping others share ideas.

Activity Instructions:

1. Introduce the Reflection

- Say: "Let's look back on everything we've explored this lesson. You learned how it feels to face communication challenges, and how tools can help people share their thoughts in special ways."

2. Guide Students Through the Checklist.

- Read each statement aloud one at a time.
- After each one, ask students to give a thumbs up, sideways, or down to show how confident they feel.
- Example prompts:
 - "Did you learn how people share ideas without talking?"
 - "Do you know how tools can help people talk?"

3. Student Completion

- Let students check off or color the "Now I Can" boxes they feel good about.
- Remind them: "It's okay if you didn't check all of them. We're all still learning."

4. Optional Extension

- Ask students to circle the one they're most proud of.
- Invite them to write a sentence or draw a picture to show their favorite learning moment.

Relation to Standards

1. Next Generation Science Standards (NGSS):

- **MS-ETS1-1:** Define problems that can be solved through engineering design.
- **MS-ETS1-2:** Evaluate competing design solutions based on criteria and constraints.

2. Common Core State Standards (CCSS) – English Language Arts:

- **CCSS.ELA-LITERACY.SL.6.1:** Engage effectively in collaborative discussions with

diverse partners.

- **CCSS.ELA-LITERACY.W.6.2:** Write informative/explanatory texts to convey ideas clearly.

3. International Society for Technology in Education (ISTE):

- **Empowered Learner:** Students use technology to set goals, work toward achieving them, and demonstrate learning.
- **Computational Thinker:** Students develop and employ strategies for solving problems leveraging technology.



Mission Innovate

Final Mission – Design for Global Communication.

Overview:

In this culminating, open-ended project, students will design a device that helps people from different language or ability backgrounds communicate clearly—especially in places where quick and effective understanding is essential (like airports, hospitals, or schools). The goal is to solve real-world language and accessibility barriers using smart design, coding, and creativity.

Project duration: 2 weeks

Teacher's Role:

- **Facilitator of Inquiry**
 - Begin by reading the mission prompt aloud. Spark curiosity by asking: “What makes global communication hard? How could tech help us speak without words?”
 - Support students as they brainstorm challenges related to language barriers and hearing/speaking difficulties.
- **Encourager of Empathy and Design Thinking.**
 - Guide students to explore different real-life scenarios where communication is difficult.
 - Encourage them to research tools like picture signs, sound/light feedback, or apps used by people with hearing or speech challenges.
- **Supporter of Student-Led Innovation.**
 - Allow students to take charge in designing a tech-based solution. This may include prototyping with cardboard, designing in MIT App Inventor, or sketching a screen interface.
 - Remind them to consider their end users’ needs—make it simple, clear, and kind.
- **Guide for Reflection**
 - Prompt students to reflect using questions like:

1. “Would this help someone feel understood?”
2. “Is the message clear without needing to speak?”

- Encourage peer feedback and self-evaluation.

- **Assessment Guide**

- Use a rubric to evaluate:
 1. Clear identification of the problem and user need.
 2. Creativity and relevance of solution.
 3. Functionality or clarity of the prototype or mockup.
 4. Communication of idea (poster, video, demo).
 5. Effort and collaboration.

Implementation Tips:

- Allow students to work in teams or pairs for collaboration and peer feedback.
- Provide tools such as:
 - Craft materials (paper, markers, LEDs, buzzers).
 - Devices with App Inventor or Tinkercad.
 - Access to internet for researching translation tools or accessibility features.
- Use Excalidraw or paper to sketch their app or device interface.
- Encourage testing with peers who pretend not to speak the same language to simulate the need.
- Dedicate time for students to share their projects through a gallery walk, video pitch, or live demo.